

Distributed Job Scheduler (Production-Grade)

version	1.0.0
license	MIT
Node.js	v20+
PostgreSQL	15+
Redis	7+

Live Demo: <https://pink-bats-bow.loca.lt> (<https://pink-bats-bow.loca.lt>)

A highly scalable, robust, and production-ready **Distributed Job Scheduling Platform**. Designed to handle massive workloads, this system allows organizations to enqueue, schedule, monitor, and execute millions of background tasks asynchronously with high reliability.

Core Features

- **Distributed Architecture:** Independent Backend API and Worker nodes scaling horizontally.
- **Atomic Job Claiming:** Redis Distributed Locks (`SET NX PX`) ensure a job is NEVER processed twice.
- **Dead Letter Queue (DLQ):** Poison-pill jobs are securely moved to a DLQ after max retries.
- **Cron & Delayed Execution:** Schedule jobs for the future or on recurring cron expressions.
- **Real-Time Monitoring:** WebSockets (Socket.io) stream live job statuses to the dashboard.
- **Beautiful UI:** A stunning React/Vite dashboard with dynamically responding **Light & Dark (Vampire/Sith) Modes**.

★ Bonus Features Implemented

1. **Workflow Dependencies (DAG):** Jobs can trigger downstream dependent jobs upon completion.
2. **AI Failure Summaries:** Integrates gracefully with LLMs to summarize *why* a job failed based on stack traces.
3. **Queue Sharding:** Dynamic queue sharding logic in Redis for high-throughput partitioning.
4. **Rate Limiting:** Redis Token-Bucket rate-limiting per individual queue.
5. **Role-Based Access Control (RBAC):** Organization-level isolation and strict JWT Role enforcement.
6. **Configurable Retry Backoffs:** Exponential backoff algorithms for network/transient failures.
7. **Global Distributed Locks:** Prevents race conditions across hundreds of worker nodes.
8. **Event-driven Webhooks:** Out-of-the-box support for triggering HTTP Webhooks.

System Architecture

```
graph TD;
  Client[Web Dashboard] -->|REST API / JWT| API[Backend Node API];
  Client <-->|WebSockets| API;
  API -->|PostgreSQL| DB[(Jobs/Users DB)];
  API -->|Redis Pub/Sub & Keys| Cache[(Redis)];
  Cache -->|Polling/Sub| Worker1[Worker Node 1];
  Cache -->|Polling/Sub| Worker2[Worker Node 2];
  Worker1 -->|Update Status| DB;
  Worker2 -->|Update Status| DB;
```

Tech Stack

Frontend

- React 18 + Vite 6
- TypeScript
- Zustand (State Management)
- React Router
- CSS Variables (Dynamic Theming)

Backend & Worker

- Node.js + Express.js
- TypeScript
- PostgreSQL + Knex.js
- Redis + ioredis
- Socket.io (WebSockets)
- Winston (Log Rotation)

How to Run Locally

1. Install Dependencies

This project is a Monorepo. Install everything from the root:

```
npm install
```

2. Environment Setup

Ensure your local **PostgreSQL** and **Redis** servers are running.
Create a `.env` in `packages/backend`:

```
PORT=5000
DATABASE_URL=postgres://postgres:postgres@localhost:5432/djs
REDIS_URL=redis://localhost:6379
JWT_SECRET=super_secret
```

3. Run Migrations & Start

Start the entire **stack** (API, Worker, Frontend) with one command:

```
npm run start
```

The UI will be instantly available at `http://localhost:5173`.

Dashboard Preview

The dashboard includes full support for system metrics, job creation, dead letter queue management, and a beautiful custom-built dark mode.

The UI utilizes CSS custom properties to rapidly switch between a pastel light mode and a deep crimson dark mode, with fully transparent inputs and glassmorphism elements.

Built for scale, reliability, and speed.